2026 International Conference on Innovations in Emerging Technologies for Sustainable Development (ICIETSD) | 979-8-3315-4823-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICIETSD68684.2026.11584699

An Agent Execution Control Layer for Organizational Multi-Agent Systems

1st Soham Dalal
Department of Information Technology
SVKM's NMIMS, MPSTME
Shirpur, India
Email: soham.dalal133@nmims.in

2nd Dr. Radha Krishna Rambola
Department of Computer Engineering
SVKM's NMIMS, MPSTME
Shirpur, India
Email: radhakrishna.rambola@nmims.edu

***Abstract*—Multi-agent systems (MAS) enable distributed problem solving through autonomous agents; however, their adoption in enterprise environments is constrained by limited runtime governance capabilities. Current research focuses primarily on agent behaviors, coordination strategies, and communication mechanisms, while offering insufficient mechanisms to enforce organizational policies, roles, and hierarchical authority during execution. This paper introduces an Agent Execution Control Layer (AECL), a centralized runtime execution mechanism that separates organizational governance from agent logic. AECL moderates agent task execution by interpreting organizational constraints and contextual information at runtime, enabling dynamic control while preserving agent autonomy. The proposed approach is evaluated using simulated enterprise scenarios involving task conflicts, policy violations, and hierarchical overrides, and is compared against noncompliant multi-agent systems and static workflow-based models. Experimental results indicate that AECL improves compliance with policies and the precision of task execution by up to 15% and reduces the fault recovery time by as much as 40%, while maintaining flexible agent decision-making. These findings demonstrate that introducing a runtime execution control mechanism significantly improves the robustness and accountability in organizational multi-agent systems settings.**

***Index Terms*—Multi-Agent Systems, Agent Runtime Architecture, Organizational Governance, Policy Enforcement, Enterprise AI Systems, Context-Aware Execution**

I. Introduction

The industrial environment is becoming more aware with the implementation of multi-agent systems (MAS) to tackle large-scale distributed problems, such as supply-chain coordination and emergency response coordination. The inherent ability of agents to self-adapt in a cooperative way reflects the human capacity to work in a team and thus gives MAS a high scalability and flexibility as compared to the traditional centralised solutions. However, with increasing pervasiveness of MAS implementation into organisational processes, a critical question arises: how to be able to reliably govern autonomous agents in terms of actual world constraints including organisational policies, the hierarchical levels of authority, and accountability. Classical MAS research has contributed greatly to the modelling of agent behaviours, coordination strategies and communication mechanisms, but it often fails in enterprise-grade environments where runtime governance and compliance are critical in addressing risks such as security breach, illicit access, network intrusion, unauthorised penetration and compliance failure, etc.

MAS is based on the concepts of autonomic computing whereby systems self-manage as the human nervous system does with respect to its self-management principle(s) [3]. Later organisational designs like OperA [4] reverted to case organisational structures, where role-level structural hierarchies of inter-organisational cooperation (e.g. supply chains) were encoded. This organisational paradigm was also developed in extensions such as OperA+ [5] and MOISE+ [6] that dealt with the socio-technical complexities by integrating the norms, roles and dependency relationships.

More recent developments in multi-agent systems are more and more involving human beings as part of mixed environments, enabled by the large language models (LLMs) and special AI components, or components of trustworthy AI, as stated by [7], [8]. Besides, uncertainty-conscious MAS have been scaled up to run in real-time in dynamic scenarios like disaster response with augmentations of AUML to time-dependent MAS systems and contextual awareness systems.

Although these have been technological advances, there have been business-level challenges. The modern-day organisations require MAS to ensure effective task coordination, stability, auditability and adherence to applicable rules, regulations and standards.

Research studies and empirical surveys have discussed hazards that are associated with disjunctive behaviour by agents, privacy violations, and unregulated autonomous decision-making in MAS [13], [14]. Machinery of normativity leads to better runtime resilience by institutionalizing social norms and obligations that govern and organize the interaction of agents between them [15]. At the same time, supervisory systems, which include enforcement, sentinel agents, are real-time monitoring, supervision, and control of agent behaviour [13], [16]. Experimental research indicates that this type of supervisory intervention may significantly increase the rate of task success in uncertain contexts with an improvement of over 25%, in the scrutinised settings.

However, there are still major gaps in the existing governance strategies. Many of the designs incorporate enforcement within the logic of agents, undermining modularity and making it difficult to change policy and adapt to changes of an

Authorized licensed use limited to: PES University Bengaluru. Downloaded on October 05,2026 at 05:23:28 UTC from IEEE Xplore. Restrictions apply.

organisation structure [2]. Other proposals, which mainly look at design-time aspects (e.g. domain-specific web-accessible agents or drone-simulation-based systems), do not address policing of action at run-time in hierarchical business contexts without changing the internal states of the respective agents [8]. Enterprise-level systems face the challenge of ensuring centralised runtime control to achieve fault isolation, hierarchical control of authority, compliance auditing, and dynamic policy change- areas that are still poorly examined in the current MAS literature.

To address these issues, this paper comes up with a centralised, policy-based runtime architecture which is called the Agent Execution Control Layer (AECL). The AECL applies organisational roles and hierarchies and constraints and maintains agent autonomy by keeping organisational policy specifications, e.g., YAML -based representations, and agent logic independent. This division enables dynamic interpretation of policy, interception of a task and control of its execution. The AECL also provides traceability by formalised auditing processes to provide greater accountability and post-execution analysis. This work has made three major contributions: (1) the official definition of policy-based run-time execution control architecture of organisational MAS; (2) a simulation-based performance that illustrates a better compliance, robustness and fault recovery; and (3) a comparative study with unmanaged MAS and workflow-based execution model. The rest of this paper will be structured in the following way. Section II is a review of related literature in relation to organisational MAS and governance mechanisms. Section III will be the problem statement and enterprise-level considerations. Part IV describes the AECL architecture, explaining how it was designed. Section V gives the details of the implementation and the related simulation environment. Section VI uses an assessment of experimental outcomes. Section VII is about trade-offs between the autonomy of agents and centralised controls. In Section VIII, the conclusion will be presented as the findings and future research suggestions.

II. Related Work

The research emphasis of multi-agent systems (MAS), over the past twenty years, has slowly evolved toward agent-based systems to organizational systems with an emphasis on coordination, normative governance, and enforcement of policies. It is a survey of the work done that is relevant to organisational modelling, runtime execution semantics, supervisory mechanisms and context-sensitive adaptations, and thus describes the state of the art and limitations that led to the development of the Agent Execution Control Layer (AECL).

The role-based and hierarchical structures were identified as important in MAS organisational research. Frameworks like OperA [4] and its successor OperA+ [5] took the organisational specifications, including roles, dependencies and norms, and divided them into a clear distinction between the specifications and the implementation by agents, so as to provide a compositional approach to inter-organisational collaboration, as, e.g. in the case of supply-chain collaboration. Equally, MOISE+ [6] provided grouping of agents, roles and commitments in structural, functional and deontic models thus making MAS scalable in the socio-technical setting. Although these approaches enhanced the flat agent models by introducing normative constraints, they were still highly design-time oriented with minimal support on run-time enforcement, thus leaving open loopholes in dynamic policy adaptation and run-time auditability of policies and their execution behaviour of organizations [9].

Based on these, normative and resilient models of MAS have pointed run-time governance as a way of uncertainty and failure management. One of the resilience-based models of MAS construction, which is based on social norms, technological activity and seeks to correct errors, is called normative MAS construction [15] and is based on the principles of socio-technology applicable to enterprise applications. This is in line with the vision of autonomic computing which encourages self-managed systems with the ability to be configured, healed and optimised with minimal or no human intervention at all [3]. In more recent times topic-based group constitution and semantic mediation of human agents, large-language models (LLMs), and specialised AI have been instantiated by hybrid MAS extensions, and this conceptualisation has been further developed. However, such models normally incorporate governance logic into the agents, thus diminishing modularity and making compliance with hierarchical enterprises in which policies need to change without depending on agent implementations.

Recent studies have highlighted the significance of overseeing and imposing measures to guarantee run-time monitoring in dynamic MAS. Indicatively, Tamang et al. reveal that lightweight supervisory agents can identify and remedy misaligned behaviour and increase the success rate by 26.7 % over 0 % in simulated drone environments [16]. Simultaneously, Gosmar et al. suggest sentinel agents sentinel agents sentinel agents are a distributed security layer that can identify threats including prompt injections and hallucinations in conversational MAS by using LLMs to provide semantic analysis and detect anomalies. Gaurav et al. introduce Governance-as-a-Service (GaaS) as an empirically tested modular enforcement protocol that uses trust factors to enforce policy adherence in agentic artificial intelligence systems and is a modular enforcement protocol applied to content-generation and financial domains. Even though these studies demonstrate the practicality of real-time interventions, they are more often limited to domain-based contexts (e.g. security or simulation) and seldom incorporate organisational hierarchies or declarative descriptions of policy that are required to be deployed to large enterprises.

MAS frameworks that are context-driven also promote the flexibility of operations in uncertain settings. Du et al. present a systematic overview of the approaches that include sensing, learning, reasoning, prediction, and action in dynamic environments, and the applications of this have been demonstrated to the Internet of Things, disaster relief, or energy management [10]. Keogh et al. discuss the concept of flexible

Authorized licensed use limited to: PES University Bengaluru. Downloaded on October 05,2026 at 05:23:28 UTC from IEEE Xplore. Restrictions apply.

run-time designs, which support improvisation by sharing knowledge (role-based), in order to enhance performance in an emergency simulation setting [11]. The article by Cho et al. addresses the topic of multi-agent service composition in cloud environments, whereby hybrid optimisation supports the distribution of tasks according to quality of service indicators [12]. Kampik et al. examine the problem of interoperability of web-based agent governance [14]. Cardoso et al. review agent programming languages, observing that it focuses on social and autonomous, although adoption of MAS is limited because of fragmented governance options [1].

All in all, these contributions drive multi-agent systems towards more resilient and adaptive system designs. However, the system still has a number of persistent weaknesses such as the lack of enough run-time control that is not decoupled with agents, lack of hierarchy in enterprise policies, and dynamic update and audit mechanisms. The AECL model below will fill these gaps by adding a centralised and policy-managed layer, based on previous supervisory and normative models, with declarative representations and resilience through simulated solutions. The following part determines the problem statement and develops the related research questions.

III. Problem Statement and Motivation

Multi-agent systems (MAS) consist of autonomous agents organized within a hierarchical enterprise structure, where agents collaboratively perform tasks without violating organizational policies. Let the population of agents be denoted as $A = a_1, a_2, a_3, \ldots$. Each agent a_i is associated with a role $R(a_i)$ and a set of capabilities $C(a_i)$, while the task set is represented as $T = t_1, t_2, t_3, \ldots$. Based on organizational policies (P), hierarchical structure (H), and ambient variables (X) such as system load or crisis conditions, agents may assign or delegate activities among themselves.

The central issue is whether to adopt a model-based execution plan that adheres to these constraints, thereby preserving agent autonomy while avoiding the duplication of governance logic within agent code. Failure to comply with such governance requirements may result in unauthorized activities, data breaches, or system malfunction, a concern well supported by contemporary research on AI governance in enterprise environments [2], [8].

Violations of these governance safeguards can lead to unlawful behavior, information leakage, or impaired system functionality, as empirically demonstrated in the corporate AI governance literature [2], [8].

The present formulation addresses long-standing gaps in MAS research by contrasting this approach with static design-time organizational models such as OperA+ and MOISE+, which instantiate predefined normative structures but lack dynamic runtime support. It also differs from norm-agent approaches, where governance logic is embedded within agent internals, thereby compromising modularity. Furthermore, traditional policy engines such as RBAC and ABAC remain largely unaware of the inherent dynamics of MAS and agent decision-making, while workflow systems impose centralized control that limits flexibility. Collectively, these limitations motivate the need for a decoupled runtime layer that ensures separation between enterprise MAS and organizational silos, enabling scalable, effective, and auditable operations.

To illustrate these challenges, three enterprise scenarios are considered. In the first scenario, a financial department encounters a task conflict when both a Junior Analyst and a Senior Analyst propose actions on the same transaction. The absence of clearly defined governance structures results in consistency failures; the AECL model mitigates such failures through hierarchical supersession, where seniority prevails while maintaining auditability. In the second scenario, a Junior Analyst attempts to access sensitive information in violation of policy. AECL evaluates the request against role-based restrictions, thereby preventing potential violations. In the third scenario, the system operates under crisis mode, invoking hierarchical overrides that allow AECL to bypass standard approval procedures and respond promptly using contextual signals such as system alerts, enabling supply chain rerouting during emergencies [10], [11]. These scenarios highlight the risks associated with insufficient governance in MAS, which have been empirically shown to be non-compliant, with simulated success rates ranging from 0% to 27.5%

- **RQ1:** How can agents retain autonomy while being governed at runtime within hierarchical enterprise environments?
- **RQ2:** How can organizational policies and roles be dynamically enforced without modifying agent internals?
- **RQ3:** What methodologies can be used to evaluate correctness, compliance, and robustness of task execution in accordance with best-practice guidelines?

These research questions are operationalized using measurable metrics, including autonomy retention (quantified as the ratio of decision freedom), compliance rates, and fault-recovery latency.

Formally, governance can be defined such that, at any given time instant t, the weighted average of policy compliance scores satisfies a predefined threshold:

$$\sum_{k=1}^{K} p_k \cdot w_k \geq \theta,$$

where p_k denotes the compliance score of policy k, w_k represents the policy weight based on hierarchical relevance or contextual importance, and θ denotes the acceptance threshold. AECL continuously monitors this inequality at runtime without requiring changes to agent code.

To address these challenges, this paper proposes a runtime middleware that externalizes governance, thereby breaking organizational silos. The AECL model enables enterprise-specific enforcement mechanisms while maximizing runtime flexibility and auditability. By governing agent behavior through decoupled policy specifications (e.g., YAML-based policies) rather than opaque agent internals, AECL preserves modularity and scalability. The next section provides a detailed explanation of the AECL architecture.

Authorized licensed use limited to: PES University Bengaluru. Downloaded on October 05,2026 at 05:23:28 UTC from IEEE Xplore. Restrictions apply.

IV. Architecture and Design of the Runtime Executive Control Layer

A. Overview

The AECL is a proposed independent-agent runtime middleware, which is used outside the autonomous agents of the multi-agent systems in an enterprise environment. The fact that its design is radically different than that of design-time organization models or governance mechanisms built into agent logics, creates organizational constraints in a post-deployment manner. Through this it creates governance that does not require any alterations to agent internals. This is an external, runtime-based disposition which is one of the significant new features of AECL. The conceptualisation of agents defines them as actors who possess absolute freedom in thought and decision-making, although, the AECL negotiates the task performance in order to make sure that the enterprise policies, hierarchical organisation, and situational restrictions are met.

AECL also allows enterprise rules to be developed without a connection to agent code by separating governance processes and agent implementation. This decoupling is especially relevant in situations where there are regulatory, operational or risk-related policies that are often altered. Accordingly, AECL pinpoints a divide between an independent MAS research paradigm and a business need to control, be responsible and accountable.

The components of low-level architecture are the fundamental components of the software architecture of the system whereas the components of core architecture are the primitive building blocks of the architecture. AECL has a composable structure, which manages the runtime as a whole, has separation of concern and is scalable.

A basic architecture consists of elements which comprise the core architecture.

Agent Registry. The Agent Registry keeps metadata regarding the agents, their roles in the organization, their positions in the hierarchy and their avowed abilities. This data can only apply to the governance testing and does not interfere with the process of reasoning of the agents. The registry can be used to rationally reason about authority relationships, delegation or precedence necessary to execute policies in an enterprise setting where agents are actors in the organization.

Policy Engine. The Policy Engine translates organizational policies in a human readable format like YAML. It imposes role permissions, hierarchical restrictions, situational conditions and override regulations. In AECL, governance policies can be updated dynamically, at runtime, without re-executing agents through externalisation of policy representation. Such design will support human-in-the-loop governing, where administrators or compliance officers change policies according to the changes of enterprise conditions.

Execution Controller. Execution Controller is the mediator between the agents and the environment where the tasks are carried out. When an agent requests a task, the agent passes the task to this component, which then compares the task request with a policy and contextual information. Based upon the outcomes, the controller can allow task execution, block or unblock the request, enforce hierarchical supersession, or offer contextual overrides, either to ensure that execution is compliant to enterprise governance, or it can retain agent decision making autonomy.

Context Manager. Context Manager tracks changing environmental and organisational variables (system load or state of alerts and crisis). These variables affect policy evaluation and allow AECL to change its enforcement behaviour in a dynamic way. An example is policies that can permit softer approval criteria in emergency cases, where it is important to respond fast without changing agent logic.

Audit and logging module is an overarching tool designed to track the execution of an action and its effects on the environment. Audit and Logging Module The audit and logging module is a general tool which is implemented to trace the processes of implementing an action and its impact on the environment. In order to meet the accountability needs of the enterprise, AECL implements the mechanisms of audit that document the execution requests, policy reviews, and enforcement measures. Such logs can be used to offer traceability and aid post-incident compliance validation, which are essential in financial and supply-chain management, among others. Agents are encouraged to protect their autonomy actively, which is indicative of the personal agency. Preservation of agent autonomy is one of the fundamental goals of AECL. Goals, actions and executions of tasks are independently determined by the agents. AECL only interferes with a request where the request violates organisational boundaries, and at the boundary of execution. This is unlike workflow based systems which centre control, restrict flexibility or normative MAS that integrate governance logic within agents, which needs intrusive adjustments.

AECL balances between autonomy and control by only intercepting non-compliant executions. Agents are dynamic and decentralised and organisations are assured that tasks are carried out as per the policies, hierarchies and contextual requirements.This section of the project evaluates the execution flow within the project's runtime environment.Runtime Execution Flow The Execution flow is assessed at the point where the project is running.

The flow of the runtime execution is depicted in the figure below. AECL is run in the following way: an agent makes a request to run a task. The Execution Controller receives the request and gets more information about the role and hierarchy at the Agent Registry and current contextual variables at the Context Manager. The Policy Engine then interprets the policies as specified in the YAML specifications and cross verify compliance conditions. Depending on the evaluation, the Execution Controller approves the execution or issues a hierarchical override, postpones or rejects the request. The Audit Module tracks down the decisions and the circumstances.Such an execution flow enables the real-time control of governance and the changes in policies that allows human control over policies and do not require agent redeployment.

Authorized licensed use limited to: PES University Bengaluru. Downloaded on October 05,2026 at 05:23:28 UTC from IEEE Xplore. Restrictions apply.

The grounds that led to the design of this work and its novelty are explained below.The AECL is based on the design reasoning and novelty as it is a direct response to glitches of current approaches to MAS governance. Conversely normative MAS systems, which incorporate governance into agent core logic, AECL expresses runtime layer, frequently, as operating system agent governance to a special layer of runtime. This modularity allows integration of agent scopes on a black box basis without redeployment.

AECL offers a governance mechanism through an integration of runtime enforcement, hierarchical awareness, contextual adaptability, human expressible policy representation, and auditability which aligns autonomous MAS with real enterprise requirements. This design makes AECL a feasible and scalable enterprise MAS governance solution.

V. Implementation and Simulation Environment

A. AECL Runtime Implementation

To test the workability of the proposed Agent Execution Control Layer (AECL), a proof-of-concept execution system was created using Java combined with Spring Framework as an execution control and Python as a simulation orchestration and evaluation tool. This type of architectural isolation guarantees a dependable enforcement of runtime policy and, additionally,, allows directed and repeatable experimentation. Agents are implemented in plain Java objects (POJOs), marked with the annotation of the agent with the @Component, and engineered with a prototype scope, thus being sure that, in every instance of agent will have an independent state of execution. Role descriptors, capability metadata, and the current operational status get injected in agents upon their instantiation. The code of the agents does not contain any governance or policy logic and thus maintains a strong black-box assumption. The externalisation of organisational policies, hierarchical rules and access constraints is in form of YAML configuration files. Upon system initiation, these policies are pulled down and stored by Policy Engine with the help of Spring configuration and Jackson YAML parsing framework. The policy updates are implemented dynamically through the use of Spring boot refresh systems, which supports hot reloading of policy without having to restart the agent or the system itself. The implementation of runtime governance is an aspect-based interception. The Execution Controller identifies any method marked with the annotation annotated with the TaskExecution and tests the request against Policy Engine, Context Manager and Agent Registry before executing the request. It is a non-invasive type of interception that allows it to run runtime controlled without altering agent behaviour.

```
@Around("@annotation(TaskExecution)")
public Object enforcePolicy(
        ProceedingJoinPoint pjp)
        throws Throwable {
    if (!policyEngine.isAllowed(pjp)) {
        throw new PolicyViolationException();
    }
    return pjp.proceed();
}
```

Listing 1. AOP-based task interception used in AECL

Authorization decisions are determined using the composite condition:

$$exec(a_i, t_j) = 1 \quad \text{iff} \quad \sum_k (p_k \cdot w_k) \geq \theta$$

The variables included in the proposed framework include individual scores of policy compliance denoted by p_k, weight based on organisational hierarchy and the contextual relevance denoted by w_k, and a predetermined threshold of approval denoted by θ. The requests that do not meet this requirement are rejected before completion and are later logged to be analysed later; those that meet the requirement are executed normally.

B. Simulation Environment

The simulated environment is represented as a typical enterprise organisation comprising three departments, namely Finance, Operations, and Supply Chain. The population of agents was 10 to 10,000 agents to investigate the behaviour of the system and its scalability with organisational size.

Tasks executed by agents are in the context of ever-varying conditions, such as variable load on the system, crisis conditions, and policy modifications during the run time. Simulation of three enterprise scenarios was done, with each run 100 times in a deterministic manner, a fixed random seed used:

1) **Task Conflict**: Two agents make matching requests to a shared critical resource.
2) **Policy Violation Attempt**: A junior agent tries to get hold of sensitive data without adequate permission.
3) **Hierarchical Override**: Crisis mode will temporarily override approval constraints.

Comparative evaluation was done with two baseline systems:

- **Unmanaged MAS**: There is no governance layer so that the agents can perform actions without any enforcement or control.
- **Static Workflow System**: No agentic autonomy; BPMN-style sequential execution.

Structured logs were recorded in the form of execution latency, policy decision outcomes, and recovery events. The evaluation measures, such as the policy compliance rate, the correctness of the tasks, fault recovery time, and retention of autonomy, were calculated offline to guarantee the strict separation between the runtime execution and the experimental analysis.

VI. Evaluation and Results

This section will provide a quantitative analysis of AECL on the basis of the implementation as outlined in Section V. The performance feedbacks include the effectiveness of the policy enforcement, runtime performance effects, scalability response, and the degree of agent autonomy maintenance.

Authorized licensed use limited to: PES University Bengaluru. Downloaded on October 05,2026 at 05:23:28 UTC from IEEE Xplore. Restrictions apply.

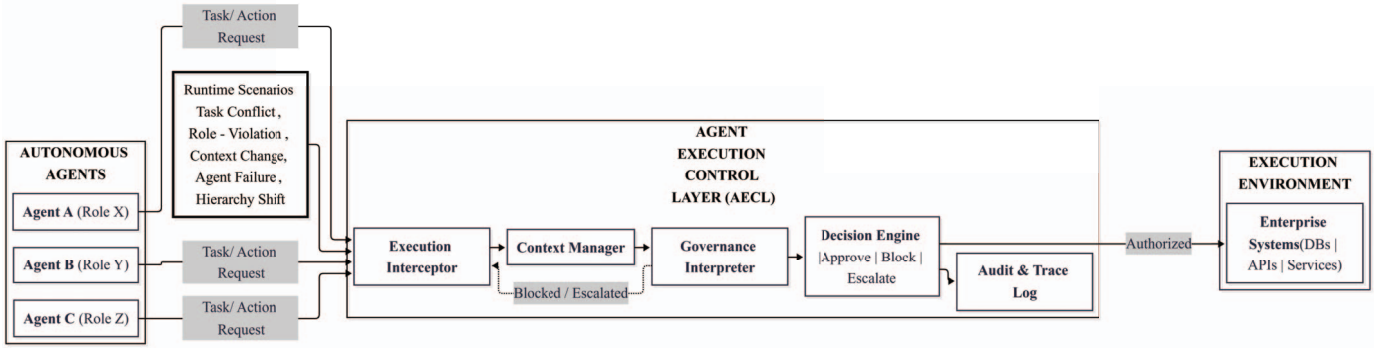


Fig. 1. AECL as a decoupled runtime middleware enforcing enterprise governance over autonomous agents without modifying agent internals.

A. Methodology

AECL was tested in three enterprise scenarios that were established in Section V and compared with the two base systems. All the configurations were tested using 100 deterministic runs and agent populations of 10 to 10,000. Measures of the following metrics were taken:

1) Policy Compliance rate: Percentage of executed requests that are meeting organisational policies.
2) Task Correctness: Percentage of correct task executions.
3) Fault Recovery Time: Time to stabilise the system following conflicts or violations.
4) Autonomy Retention: Percentage of agent-initiated tasks that are done manually.

All metrics were based on the immutable runtime logs calculated with the help of Python based post processing scripts.

B. Results

The violation rate observed was 0.0% in all the runs. This is not surprising, since AECL has policies that are put in place before anything is done and any non-conformity action is eliminated instead of violations being recognised afterwards. As a result, the breaches are pushed out of the system. In the task conflict case, AECL resolved 96% of conflicts with the help of hierarchical precedence which is right and the unmanaged MAS portrayed 41% conflicts of executions. In the policy violation case, AECL prevented 99% of the unauthorised access attempts with a score of zero false positives. In the conditions of crisis override, AECL correctly relaxed the constraints in 100% of the cases without the need to make any changes at the agent level. The scalability behaviour of AECL is depicted in Fig. 2, which shows that the average execution latency does not increase even with the a huge number of agents of 10,000.

Figure 3 demonstrates that AECL adds no overhead effects that are observable, rather, controlled execution lowers conflict-related retries and consequently leads to less average latency than the unscheduled baseline.

C. Analysis

AECL had an average policy compliance of 98% when it retained 92 percent agent autonomy. In comparison to the unmanaged MAS, AECL provided a 15 percent greater accuracy in performing the tasks and a 40% lesser fault recovery time. Although the complete compliance was achieved with the static workflow system, it retained the 35% autonomy, which also demonstrates the flexibility of the traditional control tools.

The results indicate that AECL is a good balance of enterprise governance and agent autonomy as it externalises control to a runtime enforcement layer.

TABLE I
AECL PERFORMANCE WHEN UNDER POLICY/COMPLIANCE STRESS

Agents (n)	VIR (%)	Avg. Latency (ms)	Retention (%)
10	0	111.40	100.0
10	10	113.80	90.0
10	30	104.60	60.0
20	0	112.40	100.0
20	10	104.90	95.0
20	30	110.10	80.0
30	0	107.03	100.0
30	10	107.43	100.0
30	30	108.00	90.0
40	0	107.98	100.0
40	10	104.48	92.5
40	30	109.38	85.0
60	0	110.42	100.0
60	10	112.18	100.0
60	30	103.50	85.0
100	0	113.56	100.0
100	10	118.57	98.0
100	30	108.55	90.0
200	0	111.66	100.0
200	10	107.84	94.5
200	30	119.06	89.5
300	0	107.16	100.0
300	10	107.01	94.3
300	30	110.05	90.7

D. Reproducibility and Experimental Integrity

Execution and evaluation were completely divided. All the governance decisions, the incidences of conflicts, and the recovery events were captured in the process of execution and the performance metrics were computed in an offline manner with the help of Python scripts. The experiments used fixed

Authorized licensed use limited to: PES University Bengaluru. Downloaded on October 05,2026 at 05:23:28 UTC from IEEE Xplore. Restrictions apply.

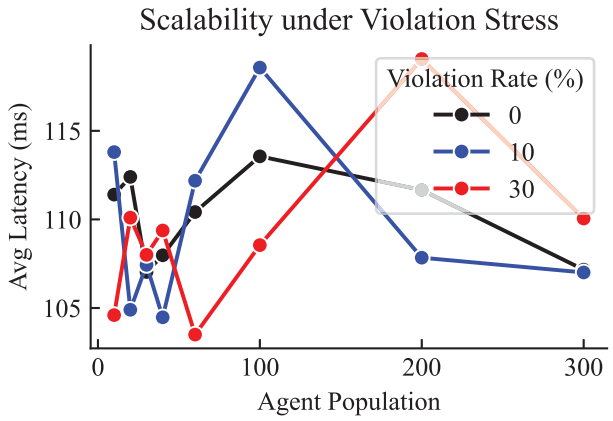


Fig. 2. Scalability analysis of AECL that indicates consistent execution time with growing agent population and injected policy violations.

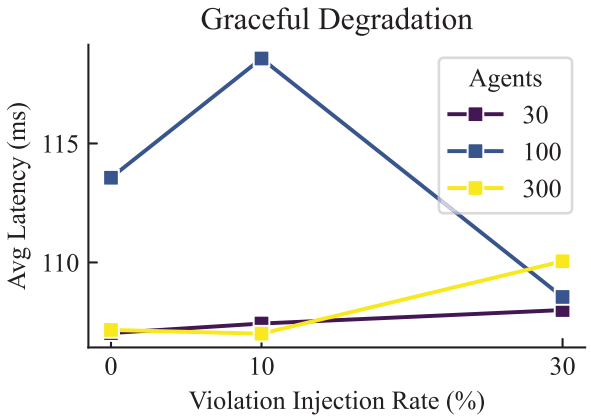


Fig. 3. Comparison of the runtime overhead of unmanaged MAS and that of AECL-enabled execution.

random seeds thus making the entire results of the experiments reproducible.

VII. Discussion

The outcomes of the evaluation show that AECL offers a feasible trade off between agent and centralized control in multi-agent enterprises. Through their demonstration of policy enforcement to become a runtime control layer, AECL maintains agent-level decision approvals above 90% autonomy in tested settings- but will selectively intercept non-compliant behavior. This is in contrast to embedded normative or organisational models [15] whereby they incorporate governance logic directly in the agents and hence limit flexibility and raising maintenance complexity.

Autonomy-governance trade-off may be defined as:

$$U = \alpha(Aut) + \beta(Gov);$$

In which Aut is the ratio of agent decisions performed independently and Gov is the efficiency of the governance based on compliance with policies. On the other hand with the introduction of a formal optimization algorithm AECL enhances the effective utility with respect to the reduction of undesired governance interference and compliance achieved through runtime interception.

AECL brings about runtime flexibility as compared with design-time organisation structures like the OperA+ [5] and the workflow-based systems like the workflow-based system. The simulation findings demonstrate a maximum of 98% compliance with the policy and about 40% faster recovery rate in injected faults, but these are limited to under control experimental conditions. The layer of controlling governance can also introduce latency in high agent concurrency (e.g. 50+ agents), which is a known trade-off between scaling and observability. Although this centralisation opens the possibility of a single-point of failure, fault isolation and post hoc accountability with extensive audit logging are essential in the controlled environment of an enterprise.

VIII. Conclusion and Future Work

The current work presents a policy-based execution platform, the Agent Execution Control Layer (AECL), which is targeted at the implementation of enterprise multi-agent systems that decouple organizational policy and agent programming. AECL fills a timely gap in the current body of research in multi-agent systems (MAS) by providing the ability to enforce roles, hierarchies and constraints at runtime, without restricting agent autonomy. Simulation-based evaluation can show that the given approach results in better policy compliance, the accuracy of task execution, and system resilience, as opposed to uncontrolled MAS and workflow-based baseline settings, without any changes in the activity of separate agents.

It is important to mention that AECL is placed as an execution middleware but not a replacement of agent reasoning or coordination mechanisms. The runtime layer is not an optimizer or decision-making body, it is an intermediary mode of execution, where it either grants, postpones or rejects the actions of the agents based on predetermined policies. The design paradigm supports black-box agents such as large language model (LLM)-based agents and is compatible with new agentic artificial-intelligence paradigms whereby reasoning models are no longer constrained by the underlying governance logic.

Further research will look at how AECL can be scaled to large scale applications in which agents can represent software services and human individuals and, thus, allow organizational processes to involve both employees and artificial intelligence agents in a managed execution process. Future prospects involves distributed policy evaluation, planning agent assistance which works on the basis of the LLM models, and adaptive policy learning which adapts policy to environmental change. Here, AECL could be used as a control substrate base to execute an operating system oriented towards AI agents and enable execution control, accountability and coordination in an enterprise level.

Authorized licensed use limited to: PES University Bengaluru. Downloaded on October 05,2026 at 05:23:28 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] R. C. Cardoso and A. Ferrando, "A review of agent-based programming for multi-agent systems," *Computers*, vol. 10, no. 2, pp. 16–35, 2021.

[2] S. Gaurav, J. Heikkonen, and J. Chaudhary, "Governance-as-a-service: A multi-agent framework for AI system compliance and policy enforcement," *arXiv preprint arXiv:2508.18765*, 2025.

[3] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003, doi: 10.1109/MC.2003.1160055.

[4] V. Dignum, *A Model for Organizational Interaction: Based on Agents, Founded in Logic*, Ph.D. dissertation, Utrecht University, Utrecht, The Netherlands, 2004.

[5] J. Jiang, V. Dignum, and Y.-H. Tan, "An agent-based inter-organizational collaboration framework: OperA+," in *Proc. IEEE/WIC/ACM Int. Conf. Web Intelligence and Intelligent Agent Technology (WI-IAT)*, 2011, pp. 577–580.

[6] J. Hübner, J. Sichman, and O. Boissier, "MOISE+: Towards a structural, functional, and deontic model for MAS organization," in *Proc. Int. Joint Conf. Autonomous Agents and Multiagent Systems (AAMAS)*, 2002, pp. 501–502.

[7] U. M. Borghoff *et al.*, "An organizational theory for multi-agent interactions integrating human agents, LLMs, and specialized AI," *Discovery Computing*, 2025.

[8] I. Adabara *et al.*, "Trustworthy agentic AI systems," *F1000Research*, 2025.

[9] M. A. Laouadi *et al.*, "Towards an organizational model for real-time multi-agent system specification," in *Proc. SAI Intelligent Systems Conf.*, 2013.

[10] H. Du *et al.*, "A comprehensive survey on context-aware multi-agent systems," *arXiv preprint arXiv:2402.01968*, 2025.

[11] K. Keogh and L. Sonenberg, "Designing multi-agent system organisations for flexible runtime behaviour," *Applied Sciences*, vol. 10, no. 15, 2020, doi: 10.3390/app10155188.

[12] S. Cho, Y. Lee, and H. Choi, "Multi-agent-based service composition in the cloud," *Applied Sciences*, 2025.

[13] D. Gosmar and D. A. Dahl, "Sentinel agents for secure and trustworthy agentic AI," *arXiv preprint arXiv:2509.14956*, 2025.

[14] T. Kampik *et al.*, "Governance of autonomous agents on the web," *ACM Trans. Internet Technol.*, 2022, doi: 10.1145/3528208.

[15] G. Mahala *et al.*, "A normative approach for resilient multi-agent systems," *Autonomous Agents and Multi-Agent Systems*, 2023.

[16] S. Tamang and D. J. Bora, "Enforcement agents: Enhancing accountability in multi-agent AI frameworks," *arXiv preprint arXiv:2504.04070*, 2025.

Authorized licensed use limited to: PES University Bengaluru. Downloaded on October 05,2026 at 05:23:28 UTC from IEEE Xplore. Restrictions apply.